

Princípio da Substituição de Liskov (LSP) e Princípio da Segregação de Interfaces (ISP)

Introdução

Nas aulas anteriores, solidificamos as bases da coesão com o SRP (Responsabilidade Única) e da extensibilidade com o OCP (Aberto/Fechado). Nesta aula, avançaremos para as regras que regem a Herança e a definição de Contratos (Interfaces) na Programação Orientada a Objetos.

O uso indiscriminado da herança é uma das maiores causas de fragilidade em sistemas complexos. Para garantir que o polimorfismo funcione de maneira segura e previsível, introduzimos os princípios de **Liskov (LSP)** e de **Segregação de Interfaces (ISP)**.

1. Princípio da Substituição de Liskov (LSP)

Formulado pela cientista da computação Barbara Liskov em 1988, este princípio define a regra de ouro para a tipagem comportamental (subtipagem):

"Se $q(x)$ é uma propriedade demonstrável dos objetos x de tipo T , então $q(y)$ deve ser verdadeiro para objetos y de tipo S , onde S é um subtipo de T ."

Em termos práticos para a Engenharia de Software: **As classes derivadas (filhas) devem poder substituir as suas classes base (pais) sem alterar a corretude do programa.** Se um método espera receber um objeto do tipo A , ele deve ser capaz de operar perfeitamente recebendo uma instância de B (onde B extends A), sem precisar verificar o tipo da instância e sem que o sistema lance exceções inesperadas.

Abaixo, analisaremos três cenários distintos que ilustram violações e aplicações deste princípio.

Exemplo 1.1: O Paradoxo do Quadrado e do Retângulo (Geometria)

Este é o exemplo clássico da literatura. Matematicamente, um quadrado é um retângulo. Na modelagem orientada a objetos, essa herança quebra o comportamento esperado.

O Antipadão (Violação do LSP):

Se Quadrado herda de Retangulo, ele precisa garantir que os lados sejam iguais. Ao alterar a largura, altera também a altura. Isso quebra a expectativa de qualquer código que use a classe Retangulo base, que espera que largura e altura sejam independentes.

Java

```
public class Retangulo {
    protected double largura;
    protected double altura;

    public void setLargura(double largura) { this.largura = largura; }
    public void setAltura(double altura) { this.altura = altura; }
    public double getArea() { return largura * altura; }
}

public class Quadrado extends Retangulo {
    @Override
    public void setLargura(double largura) {
        this.largura = largura;
        this.altura = largura; // Força a igualdade, quebrando o comportamento base
    }

    @Override
    public void setAltura(double altura) {
        this.altura = altura;
        this.largura = altura;
    }
}
```

A Solução Arquitetural:

Quadrados e Retângulos não devem herdar um do outro, pois seus comportamentos de mutação de estado são incompatíveis. Ambos devem implementar uma abstração comum.

Java

```
public interface FormaGeometrica {
    double getArea();
}

public class Retangulo implements FormaGeometrica {
```

```

    private double largura, altura;
    // Construtores e setters omitidos
    @Override public double getArea() { return largura * altura; }
}

```

```

public class Quadrado implements FormaGeometrica {
    private double lado;
    // Construtores e setters omitidos
    @Override public double getArea() { return lado * lado; }
}

```

Exemplo 1.2: Regras de Negócio e Bonificações (Domínio de RH)

O Antipadão (Violação do LSP):

O sistema assume que todo Funcionario recebe bonificação. Quando a empresa contrata um Voluntario ou Terceirizado (que legalmente não pode receber bônus), o desenvolvedor anula o método lançando uma exceção. Um processo em lote que calcule a folha de pagamento irá falhar catastroficamente ao processar essa instância.

Java

```

public class Funcionario {
    protected double salario;

    public void aplicarBonificacao(double valor) {
        this.salario += valor;
    }
}

```

```

public class Voluntario extends Funcionario {
    @Override
    public void aplicarBonificacao(double valor) {
        throw new UnsupportedOperationException("Voluntários não recebem bonificação.");
    }
}

```

A Solução Arquitetural:

Segregar os tipos de vínculos e usar interfaces para determinar quem é elegível a benefícios financeiros.

Java

```
public abstract class Colaborador {  
    private String nome;  
    private String documento;  
}
```

```
public interface ElegivelBonificacao {  
    void aplicarBonificacao(double valor);  
}
```

```
public class FuncionarioCLT extends Colaborador implements ElegivelBonificacao {  
    private double salario;  
    @Override  
    public void aplicarBonificacao(double valor) { this.salario += valor; }  
}
```

```
public class Voluntario extends Colaborador {  
    // Não implementa a interface, logo, o sistema nunca tentará bonificá-lo por engano.  
}
```

Exemplo 1.3: Permissões de Arquivos (Infraestrutura)

O Antipadão (Violação do LSP):

Um sistema gerencia documentos. A classe Documento permite leitura e escrita. Cria-se então um DocumentoSomenteLeitura que herda de Documento, mas bloqueia a escrita.

Java

```
public class Documento {  
    protected String conteudo;
```

```

    public void ler() { System.out.println(conteudo); }

    public void escrever(String texto) { this.conteudo = texto; }
}

public class DocumentoSomenteLeitura extends Documento {
    @Override
    public void escrever(String texto) {
        throw new IllegalStateException("Este documento é protegido contra escrita.");
    }
}

```

A Solução Arquitetural:

A capacidade de escrita deve ser extraída. A classe base (ou interface) deve garantir apenas o comportamento comum e irrefutável (leitura).

Java

```

public abstract class Documento {
    protected String conteudo;
    public abstract void ler();
}

public class DocumentoEditavel extends Documento {
    public void escrever(String texto) { this.conteudo = texto; }

    @Override
    public void ler() { System.out.println(conteudo); }
}

public class DocumentoSomenteLeitura extends Documento {
    @Override
    public void ler() { System.out.println(conteudo); }
    // Não possui método escrever(). O contrato não é violado.
}

```

2. Princípio da Segregação de Interfaces (ISP)

"Módulos clientes não devem ser forçados a depender de interfaces (métodos) que eles não utilizam."

Este princípio é a contraparte do SRP aplicada aos contratos. Uma "Interface Gorda" (*Fat Interface*) ocorre quando agregamos dezenas de métodos distintos num único contrato. Isso força as classes concretas a implementarem métodos com corpos vazios ou lançando exceções, gerando alto acoplamento e inevitáveis violações do LSP.

Vejamos três manifestações deste problema.

Exemplo 2.1: Dispositivos Multifuncionais (Hardware)

O Antipadão (Violação do ISP):

A interface *Maquina* exige que todo dispositivo saiba imprimir, escanear e enviar fax. Uma impressora térmica simples é forçada a carregar métodos inúteis.

Java

```
public interface Maquina {
    void imprimir(String documento);
    void escanear(String documento);
    void enviarFax(String documento);
}

public class ImpressoraBasica implements Maquina {
    @Override
    public void imprimir(String documento) { System.out.println("Imprimindo: " + documento); }

    @Override
    public void escanear(String documento) { throw new UnsupportedOperationException(); }

    @Override
    public void enviarFax(String documento) { throw new UnsupportedOperationException(); }
}
```

A Solução Arquitetural:

Dividir a interface grande em interfaces menores e coesas.

Java

```
public interface Impressora {  
    void imprimir(String documento);  
}
```

```
public interface Escaner {  
    void escanear(String documento);  
}
```

```
public interface AparelhoFax {  
    void enviarFax(String documento);  
}
```

// A classe consome apenas o contrato que faz sentido para ela

```
public class ImpressoraBasica implements Impressora {  
    @Override  
    public void imprimir(String documento) { System.out.println("Imprimindo..."); }  
}
```

// Equipamentos complexos compõem múltiplas interfaces

```
public class MultifuncionalPremium implements Impressora, Escaner, AparelhoFax {  
    @Override public void imprimir(String doc) { /* ... */ }  
    @Override public void escanear(String doc) { /* ... */ }  
    @Override public void enviarFax(String doc) { /* ... */ }  
}
```

Exemplo 2.2: Gateways de Pagamento (E-commerce)

O Antipadão (Violação do ISP):

Uma interface única para transações financeiras engloba lógica de cartão de crédito, boletos e transferências instantâneas.

Java

```

public interface GatewayPagamento {
    void pagarComCartaoCredito();
    void gerarBoleto();
    void processarPix();
}

public class ServicoPagamentoPix implements GatewayPagamento {
    @Override
    public void pagarComCartaoCredito() { /* Vazio - Não faz sentido */ }

    @Override
    public void gerarBoleto() { /* Vazio - Não faz sentido */ }

    @Override
    public void processarPix() {
        System.out.println("Processando chave PIX...");
    }
}

```

A Solução Arquitetural:

Segregar os meios de pagamento. Um cliente que deseja apenas processar PIX não precisa conhecer os métodos de cartão de crédito.

Java

```

public interface PagamentoCartao {
    void processarCredito();
}

public interface PagamentoBoleto {
    void emitirBoleto();
}

public interface PagamentoInstantaneo {
    void processarPix();
}

```



```
}
```

```
public class ServicoPagamentoPix implements PagamentoInstantaneo {  
    @Override  
    public void processarPix() {  
        System.out.println("Processando chave PIX...");  
    }  
}
```

Exemplo 2.3: Autômatos e Trabalhadores (Comportamento Genérico)

O Antipadão (Violação do ISP):

Uma interface modela o dia a dia de um trabalhador em uma fábrica. Porém, ao introduzirmos robôs na linha de montagem, a interface se mostra inadequada.

Java

```
public interface TrabalhadorFAbrica {  
    void trabalhar();  
    void comer();  
    void dormir();  
}
```

```
public class RoboMontagem implements TrabalhadorFAbrica {  
    @Override  
    public void trabalhar() { System.out.println("Montando peças..."); }  
  
    @Override  
    public void comer() { throw new UnsupportedOperationException("Robôs não comem."); }  
  
    @Override  
    public void dormir() { throw new UnsupportedOperationException("Robôs não dormem."); }  
}
```

A Solução Arquitetural:

Separar a ação de trabalhar das ações biológicas.

Java

```
public interface Produtivo {  
    void trabalhar();  
}
```

```
public interface NecessidadesBiologicas {  
    void comer();  
    void dormir();  
}
```

```
public class Humano implements Produtivo, NecessidadesBiologicas {  
    @Override public void trabalhar() { /* ... */ }  
    @Override public void comer() { /* ... */ }  
    @Override public void dormir() { /* ... */ }  
}
```

```
public class RoboMontagem implements Produtivo {  
    @Override  
    public void trabalhar() { System.out.println("Montando peças ininterruptamente..."); }  
    // Nenhuma dependência inútil foi herdada.  
}
```

Considerações Finais da Aula

A simbiose entre o **LSP** e o **ISP** é evidente: ao segregarmos nossas interfaces em contratos menores e altamente coesos (ISP), reduzimos drasticamente a chance de forçar uma classe filha a implementar um comportamento que não possui, prevenindo assim a violação da tipagem comportamental (LSP).

Sistemas corporativos robustos baseiam-se em pequenas peças bem definidas (interfaces segregadas) que podem ser substituídas transparentemente umas pelas outras (substituição de Liskov).